230547335 AB SIBAMBO

Semester Reflection Report:

The following is what I learned through the semester and the role I played in the project

Focus Area:Client-Side Routing,Controlled Forms,and RESTful POST Operations
Technologies: React, Spring Boot, Java EE 7,Axios

As our team transitioned from the tightly coupled Java EE 7 ecosystem to a modern decoupled architecture using React and Spring Boot, my role shifted toward facilitating seamless data entry and user interactions. In our prior academic modules, capturing user input meant building standard HTML forms inside JavaServer Pages (JSPs) that would trigger a full-page synchronous POST request to a Java Servlet. The Servlet would then manually extract strings via request.getParameter(), validate them, and either process a database transaction or redirect the user back to the form with error messages. For this capstone project—a car wash management system for the Witbank branch—I was tasked with modernizing this flow by engineering the Booking React component. This module served as a profound learning experience in building Single Page Applications (SPAs), managing controlled components, handling client-side routing, and executing asynchronous POST requests.

Total Makeover From Request Parameters to Controlled Components

The most fundamental shift in my thinking occurred when I began building the HTML form inside the Booking component. In a traditional Java EE application, form inputs are "uncontrolled" by default; the DOM holds the state, and the server only sees the data when the submit button is pressed. In React, I had to adopt the concept of controlled components.

I utilized the useState hook to initialize a formData object containing the plateNumber, carModel, phoneNumber, and washType. I learned that the React state must act as the "single source of truth." Every time a user types a keystroke into an input field, an onChange event fires, capturing the event target’s value and updating the state object using the spread operator (e.g., setFormData({...formData, carModel: e.target.value})).

This real-time synchronization between the UI and the underlying JavaScript object was revelatory. It allowed for instantaneous data manipulation before the data ever left the user's device. For

example, by simply appending .toUpperCase() to the plate number's onChange handler, I was able to guarantee standardized data formatting instantly. In our older Java EE projects, formatting this data would have either required writing custom vanilla JavaScript DOM manipulation scripts or waiting until the data hit the Java backend to invoke .toUpperCase() on the backend, neither of which provides the elegant, immediate user experience of React.

Enhancing User Experience via Client-Side Validation

Another critical area of learning was the implementation of client-side validation. Because the state is continuously updated as the user types, validating the data prior to submission becomes incredibly straightforward. In the handleSubmit function, before making any network calls to the Spring Boot server, I implemented a check: if (formData.phoneNumber.length < 10). If this condition fails, the browser immediately alerts the user, and the execution returns, preventing a wasteful HTTP request.

While we still implemented robust validation on our Spring Boot backend (using annotations like @Valid and @Size), handling this preliminary validation on the frontend drastically improved the application's responsiveness. It taught me a valuable lesson in enterprise architecture: the frontend should act as a rapid feedback loop for the user, while the backend serves as the final, impenetrable fortress of data integrity.

Mastering Client-Side Routing and State Passing

Working with react-router-dom fundamentally changed my understanding of web navigation. In Java EE 7, navigation is inherently server-driven. Moving from a "Services" page to a "Booking" page meant clicking a link, hitting the server, and having the server render and return a completely new JSP. Furthermore, passing contextual data between these pages often required attaching query parameters to the URL or heavily utilizing the HttpSession, which consumes server memory.

React Router allowed me to handle navigation entirely within the browser, avoiding the jarring screen flickers of traditional page loads. I utilized the useLocation hook to elegantly capture state passed from previous screens. For instance, if a user clicked "Book Now" under the "Full Valet" option on a separate pricing page, the router would pass that selection invisibly in the background. My useEffect hook would intercept this location.state?.selectedService value and automatically pre-populate the washType dropdown in the booking form. This decoupled approach to passing transient state is highly memory-efficient and creates a remarkably fluid user experience.

Similarly, upon a successful booking, I used the useNavigate hook (Maps('/tracker')) to programmatically redirect the user to the live queue board. This client-side redirect happens instantly, without requiring the Spring Boot backend to issue a bulky HttpServletResponse.sendRedirect().

Executing Asynchronous API POST Requests

The culmination of the Booking component was the handleSubmit function, where the frontend finally communicates with the backend. In Java EE, submitting a form inherently triggers a page reload. To prevent this default browser behavior, my first line of code inside the submission handler was e.preventDefault(). This single line represents the essence of modern SPAs: we take control away from the browser and handle the data submission ourselves.

I utilized the axios.post method to send the formData object to our Spring Boot REST API (http://localhost:8080/api/bookings). Because axios automatically serializes the JavaScript object into a JSON payload, the integration with Spring Boot was phenomenally smooth. On the backend, my teammates simply needed a @RequestBody annotation to deserialize the JSON straight into a Java Data Transfer Object (DTO). This seamless JSON contract completely eliminated the tedious request.getParameter() boilerplate code we used to write in our Servlets.

Furthermore, executing this asynchronously using async/await allowed me to handle the network response gracefully. By wrapping the Axios call in a try/catch block, I could capture the newly generated Ticket ID returned by the Spring Boot server and alert the user. If the backend was offline, the catch block provided a user-friendly error message without breaking the frontend interface.

Conclusion:

Reflecting on the development of the Booking component, the contrast between our previous Java EE 7 architecture and our current React/Spring Boot stack is night and day. Building this form taught me that modern frontend development is about much more than just HTML and CSS; it is about engineering a responsive, state-driven application that interacts asynchronously with distributed services.

Through mastering controlled components, I learned how to maintain data consistency in real-time. Through client-side routing, I learned how to optimize navigation and transient state management without burdening the server. And through utilizing Axios to communicate with our Spring Boot API, I gained a deep appreciation

for the clean, JSON-based decoupling of presentation and business logic. I am walking away from this semester not only with a functional capstone project but with a robust, industry-ready skill set in modern full-stack web development.